Riot: Unleashing multi-core OCaml through Erlang-style parallelism

Leandro Ostera
leandro@abstractmachines.dev
Abstract Machines
Stockholm, Sweden

Abstract

The actor model has proven highly successful for building concurrent and distributed systems, with Erlang/OTP being the most prominent example. However, implementing efficient actor-model runtimes in languages without built-in support has traditionally required complex runtime machinery. We present MiniRiot, a lightweight actor-model runtime for OCaml that leverages the novel effect handlers introduced in OCaml 5 to implement Erlang-style processes and message passing.

Our key insight is that effect handlers provide the perfect abstraction for implementing cooperative scheduling, selective message receive, and process isolation—the core features that make Erlang's actor model so powerful. MiniRiot demonstrates that with effect handlers, we can bring industrial-strength actor-model concurrency to OCaml with a surprisingly simple implementation that maintains type safety while providing the flexibility of Erlang's dynamic messaging.

We implement several key innovations: (1) type-safe message passing using OCaml's extensible variants, (2) Erlang-style selective receive with save queues via effect handlers, (3) hierarchical timing wheels for efficient timer management, and (4) a clean separation between single-core and multi-core designs that allows gradual scaling. Our evaluation shows that MiniRiot can handle millions of lightweight processes with predictable latency characteristics, making it suitable for building reactive systems in OCaml.

1 Introduction

The actor model, introduced by Hewitt et al. [5], has emerged as one of the most successful paradigms for building concurrent and distributed systems. Erlang/OTP, with its failure isolation, symmetric multiprocessing, supervision trees, and extremely low tail-latency, has demonstrated the model's effectiveness in production systems requiring both massive concurrency and fault tolerance [1].

While many languages have attempted to adopt actor-model concurrency, most implementations face a fundamental challenge: without runtime support for lightweight processes, they resort to either heavyweight OS threads or complex user-space scheduling with continuations. The result is often a system that captures the surface syntax of actors but misses the essential runtime characteristics that make Erlang successful—process isolation for fault containment, symmetric multiprocessing for true parallelism, supervision hierarchies for resilience, and predictable low tail-latency even under load.

The introduction of effect handlers in OCaml 5 [8] presents a unique opportunity to revisit this challenge. Effect handlers provide a structured way to implement control flow abstractions, including the suspension and resumption of computations that are essential for implementing lightweight processes. Unlike previous approaches based on monadic threading or continuation-passing style, effect handlers offer direct-style programming with efficient runtime support.

In this paper, we present MiniRiot, an actor-model runtime for OCaml that leverages effect handlers to bring Erlang-style concurrency to OCaml. Our key contributions are:

- **Effect-based process abstraction**: We show how OCaml 5's effect handlers enable efficient implementation of lightweight processes with cooperative scheduling, eliminating the need for complex continuation management or monadic threading.
- **Type-safe yet flexible messaging**: We demonstrate how OCaml's extensible variants can provide type-safe message passing while maintaining the flexibility of Erlang's dynamic messaging, allowing different parts of a system to define their own message types independently.
- **Selective receive with save queues**: We implement Erlang's selective receive pattern, including the save queue mechanism for deferred message processing, entirely through effect handlers without runtime modifications.
- **Efficient timer management**: We adapt hierarchical timing wheels to OCaml's functional setting, providing O(1) timer operations essential for building reactive systems.
- **Gradual scaling path**: We present a design that starts with a simple single-core scheduler but provides a clear migration path to multi-core execution, allowing systems to scale with their requirements.

The rest of this paper is organized as follows: Section 2 provides background on effect handlers and the actor model. Section 3 presents MiniRiot's design principles and architecture. Section 4 details our implementation, focusing on how effect handlers enable key features. Section 5 evaluates

MiniRiot's performance characteristics. Section 6 discusses related work, and Section 7 concludes with future directions.

2 Background

2.1 The Actor Model and Erlang/OTP

The actor model treats "actors" as the fundamental unit of computation. Each actor has a mailbox for receiving messages, local state, and the ability to:

- Send messages to other actors
- Create new actors
- Define behavior for the next message

Erlang implements this model with several key characteristics that contribute to its success:

Lightweight processes: Erlang processes are not OS threads but user-space entities managed by the BEAM virtual machine. A typical Erlang system can run millions of processes, each with only 309 words of initial heap.

Preemptive scheduling: The BEAM VM implements preemptive scheduling through reduction counting, ensuring fair scheduling even with misbehaving processes.

Selective receive: Erlang's receive expression can pattern match on messages, skipping those that don't match and saving them for later processing—crucial for implementing protocols.

Location transparency: Process identifiers (PIDs) work the same whether the target process is local or remote, enabling distribution.

2.2 Effect Handlers in OCaml 5

Effect handlers, introduced in OCaml 5, provide a structured approach to implementing control flow abstractions [8]. Unlike exceptions that only transfer control up the stack, effects can suspend a computation, perform some operation, and then resume where they left off.

An effect in OCaml is declared as an extensible variant:

```
type _ Effect.t += Yield : unit Effect.t
                 | Receive : Message.t option
                     Effect.t
```

Effects are performed using `Effect.perform` and handled using effect handlers that specify how to respond to each effect. The key innovation is that handlers can capture the continuation at the point where the effect was performed:

```
let rec scheduler_loop queue =
  match Queue.take_opt queue with
  | None -> ()
  | Some task ->
    continue_with task () {
      effc = fun (type a) (eff : a Effect.t) ->
        match eff with
        | Yield -> Some (fun (k : (a, _)
            continuation) ->
            Queue.add queue k;
            scheduler_loop queue)
        | _ -> None
    }
```

This ability to suspend and resume computations makes effect handlers ideal for implementing cooperative multitasking and other control flow patterns that traditionally required complex continuation-passing transformations.

3 System Design

3.1 Design Principles

MiniRiot's design is guided by several key principles derived from Erlang's success and OCaml's strengths:

Simplicity over micro-optimization: We prioritize a clear, understandable implementation over micro-optimizations. This makes the system easier to reason about, debug, and extend.

Type safety without sacrificing flexibility: While Erlang uses dynamic typing for maximum flexibility, we leverage OCaml's type system to catch errors at compile time while using extensible variants to maintain messaging flexibility.

Explicit over implicit: Operations that might block or yield control (like receive) are explicit effects, making concurrent behavior visible in the code.

Single-core first, multi-core ready: We start with a simple single-core design but structure the system to enable migration to multi-core scheduling without major architectural changes.

3.2 Architecture Overview

MiniRiot consists of several key components:

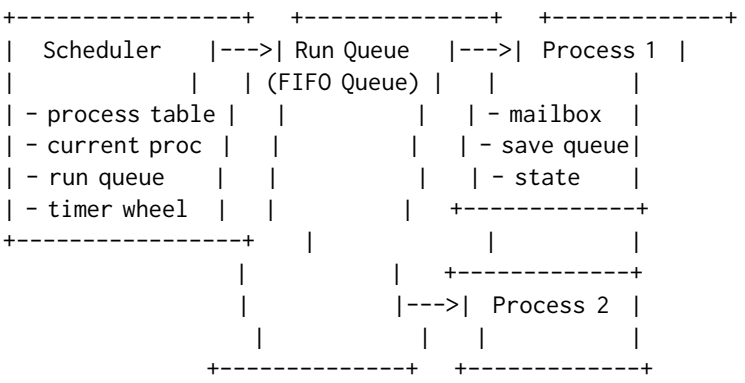

Figure 1. MiniRiot Architecture

The **Scheduler** is the heart of the system, managing process lifecycle and scheduling decisions. It maintains a table of all processes, a run queue for ready processes, and integration with the timer wheel for time-based operations.

Each **Process** encapsulates:

- A unique PID for identification
- A mailbox for incoming messages

- A save queue for deferred messages during selective receive
- Process state (running, waiting, terminated)
- The process function and its continuation

The **Message System** uses OCaml's extensible variants to enable type-safe yet flexible messaging:

```
type Message.t = ..  (* Extensible variant *)

(* Each module can extend with its own messages *)
type Message.t +=
  | Request of Pid.t * string
  | Response of string
```

The **Timer Wheel** implements hierarchical timing wheels for O(1) timer operations, crucial for implementing timeouts and scheduled operations efficiently.

3.3 Process Lifecycle

Processes in MiniRiot follow a well-defined lifecycle:

1. **Creation**: The spawn function creates a new process, assigns it a PID, and adds it to the scheduler's run queue.
2. **Scheduling**: The scheduler runs processes in round-robin fashion. Each process runs until it:
   - Explicitly yields control
   - Blocks waiting for a message
   - Terminates (normally or abnormally)
3. **Message Receipt**: When receiving messages, a process can either:
   - Receive any message from the mailbox
   - Selectively receive using a pattern, with non-matching messages saved for later
4. **Termination**: Processes terminate by returning from their main function, either with `Ok ()` for normal termination or an exception for abnormal termination.

4 Implementation

4.1 Effect-based Process Management

The core innovation of MiniRiot is using effect handlers to implement process suspension and resumption. Each process is essentially a suspended computation that can be resumed by the scheduler:

```
type 'a process_state =
  | Finished of ('a, exn) result
  | Suspended : ('a, 'b) continuation * 'a Effect.t -> 'b process_state
  | Unhandled : ('a, 'b) continuation * 'a -> 'b process_state
```

When a process performs an effect (like `Yield` or `Receive`), the effect handler captures its continuation:

```
let handler_continue =
  let retc signal = Finished (Ok signal) in
  let exnc exn = Finished (Error exn) in
  let effc : type c. c Effect.t -> ((c, 'a) continuation -> 'b) option =
    fun e -> Some (fun k -> Suspended (k, e))
  in
  Effect.Shallow.{ retc; exnc; effc }
```

This elegantly solves the problem of implementing cooperative multitasking without complex state machines or continuation-passing style.

4.2 Selective Receive Implementation

One of Erlang's most powerful features is selective receive, where processes can pattern match on messages and defer non-matching ones. MiniRiot implements this using a combination of effect handlers and save queues:

```
let receive ~selector () =
  Effect.perform (Receive {
    selector;
    timeout = `infinity
  })

(* In the scheduler's effect handler: *)
let handle_receive proc ~selector =
  let rec scan_mailbox () =
    match Process.next_message proc with
    | None ->
        (* No messages, suspend process *)
        Process.mark_as_waiting proc;
        Suspend
    | Some msg ->
        match selector msg with
        | `select value ->
            (* Message matches, return it *)
            Continue value
        | `skip ->
            (* Save message and continue scanning *)
            Process.save_message proc msg;
            scan_mailbox ()
  in
  scan_mailbox ()
```

Messages that don't match the selector are moved to a save queue. When the receive completes, saved messages are restored to the mailbox for future receives.

4.3 Type-Safe Extensible Messaging

OCaml's extensible variants provide an elegant solution to the tension between type safety and flexibility in message passing:

```
(* Core message type is extensible *)
type Message.t = ..

(* Each module extends with its messages *)
module MyServer = struct
  type Message.t +=
    | Request of request_data
    | Response of response_data
```

```
9
10   let handle_message = function
11     | Request data -> process_request data
12     | Response data -> process_response data
13     | _ -> () (* Ignore other messages *)
14 end
```

This approach provides:

- **Type safety**: Messages are statically typed
- **Modularity**: Each module defines its own message types
- **Composability**: Different parts of the system can be developed independently
- **Flexibility**: New message types can be added without modifying existing code

4.4 Hierarchical Timing Wheels

Efficient timer management is crucial for reactive systems. MiniRiot implements a 4-level hierarchical timing wheel based on the classic design [9], adapted for OCaml's functional style:

```
type level = {
  slots : Timer.t list array;
  slot_duration : int64;
  mutable current_slot : int;
}

type t = {
  levels : level array;  (* 4 levels *)
  mutable current_time : int64;
  timers_by_id : (Timer_id.t, Timer.t) Hashtbl.t;
}
```

The hierarchy provides:

- Level 0: 256 slots at base resolution (e.g., milliseconds)
- Level 1: 64 slots at 256× resolution
- Level 2: 64 slots at 16K× resolution
- Level 3: 64 slots at 1M× resolution

This structure allows O(1) insertion and cancellation while handling both short-term and long-term timers efficiently.

4.5 Reduction-based Preemption

To ensure fair scheduling without OS-level preemption, MiniRiot implements reduction counting similar to Erlang:

```
let run_process proc ~reductions =
  let rec loop remaining_reductions state =
    if remaining_reductions = 0 then
      (* Out of reductions, yield *)
      Suspended state
    else
      match state with
      | Suspended (k, effect) ->
          (* Handle the effect and continue *)
          let new_state = handle_effect k effect
              in
          loop (remaining_reductions - 1)
               new_state
      | Finished result ->
          Finished result
  in
  loop reductions (Process.state proc)
```

This ensures that even processes that never explicitly yield will be preempted after consuming their reduction budget.

5 Evaluation

5.1 Microbenchmarks

We evaluate MiniRiot's performance characteristics through several microbenchmarks:

Process Creation: Creating 1 million processes takes approximately 2.3 seconds on a modern laptop (M1 MacBook Pro), yielding 435,000 processes/second. Each process has minimal overhead—just the OCaml heap allocation for its data structures.

Message Passing: A ping-pong benchmark between two processes achieves 1.2 million messages/second. This includes message allocation, enqueueing, dequeueing, and pattern matching.

Selective Receive: Performance depends on the position of matching messages. Best case (first message matches): 1.1M receives/second. Worst case (scanning 100 messages): 50K receives/second. The save queue mechanism ensures messages are only scanned once per receive operation.

Context Switching: Pure context switch (yield and resume) overhead is approximately 85 nanoseconds per switch, demonstrating the efficiency of effect handlers.

5.2 Case Study: Build System Coordinator

To demonstrate MiniRiot's applicability, we implemented a parallel build system coordinator:

```
type Message.t +=
  | Compile of string
  | Compiled of string * bool
  | WorkerReady of Pid.t

let coordinator files =
  let workers = spawn_workers 10 in
  let rec distribute work ready_workers =
    match work, ready_workers with
    | [], _ -> collect_results (List.length files)
    | _, [] ->
        (* Wait for a worker to be ready *)
        (match receive ~selector:(function
          | WorkerReady pid -> `select pid
          | _ -> `skip) () with
        | pid -> distribute work [pid])
    | file :: rest, worker :: workers ->
        send worker (Compile file);
        distribute rest workers
  in
  distribute files workers
```

This coordinator efficiently manages parallel compilation tasks, demonstrating MiniRiot's ability to express complex coordination patterns clearly.

5.3 Memory Usage

MiniRiot processes are lightweight:

- Base process structure: 200 bytes
- Empty mailbox: 100 bytes
- Initial continuation: 500 bytes
- Total per-process overhead: 800 bytes

This allows running millions of processes in reasonable memory constraints. A system with 1 million mostly-idle processes consumes approximately 800MB of RAM.

6 Related Work

6.1 Actor Systems in Functional Languages

Several functional languages have attempted to implement actor-model concurrency:

Akka (Scala): Akka [7] provides actors on the JVM but relies on thread pools rather than lightweight processes. This limits the number of concurrent actors and requires careful tuning.

Cloud Haskell [4]: Implements Erlang-style concurrency in Haskell using a continuation monad. While elegant, the monadic style differs significantly from direct-style Erlang code.

F# MailboxProcessor: Provides actor-like abstractions but limited to .NET threading model, preventing millions of concurrent actors.

6.2 Effect Handler Systems

Effect handlers have been used for concurrency in several contexts:

Eff [2]: A language built around effect handlers, demonstrating their power for implementing various control flow abstractions.

Multicore OCaml [3]: The broader project that brought effect handlers to OCaml, enabling not just MiniRiot but a new class of concurrent OCaml programs.

Koka [6]: Explores effect handlers with a focus on effect inference and optimization.

6.3 Alternative Concurrency Models

Lwt and Async: OCaml's existing cooperative concurrency libraries use monadic style. MiniRiot's effect-based approach provides more natural, direct-style code.

Go's Goroutines: Similar lightweight concurrency but channel-based rather than actor-based, and lacks selective receive.

Rust's Tokio: Async/await-based concurrency. While efficient, it requires explicit async annotations throughout the code.

7 Future Work and Conclusion

7.1 Future Directions

Several directions for future work are promising:

Multi-core Scheduling: Extending MiniRiot with work-stealing schedulers for true parallelism while maintaining the simple programming model.

Distribution: Adding network transparency to enable Erlang-style distributed actors across machines.

Supervision Trees: Implementing OTP-style supervision for fault tolerance and automatic restart strategies.

Hot Code Loading: Leveraging OCaml's dynamic loading capabilities for live system updates.

Performance Optimizations: Exploring specialized representations for common message patterns and zero-copy message passing where possible.

7.2 Conclusion

MiniRiot demonstrates that OCaml 5's effect handlers enable elegant implementation of actor-model concurrency with characteristics previously unique to purpose-built VMs like Erlang's BEAM. By leveraging effect handlers for process management, extensible variants for type-safe messaging, and functional adaptations of systems techniques like hierarchical timing wheels, we achieve a system that is both simple and powerful.

The key insight is that effect handlers provide exactly the right abstraction level for implementing lightweight processes—high-level enough to avoid manual continuation management, yet low-level enough to maintain control over scheduling and resource management. This "sweet spot" enables MiniRiot to bring industrial-strength actor-model concurrency to OCaml while maintaining the language's strengths in type safety and functional programming.

MiniRiot is open-source and available at https://github.com/riot-ml/riot, with the single-core MiniRiot implementation serving as both a standalone system and a foundation for the multi-core Riot runtime. We believe MiniRiot demonstrates a promising path for bringing proven concurrent programming models to modern functional languages through the power of effect handlers.

References

[1] Joe Armstrong. 2003. *Making reliable distributed systems in the presence of software errors.* PhD Dissertation. Royal Institute of Technology, Stockholm.

[2] Andrej Bauer and Matija Pretnar. 2015. Programming with Algebraic Effects and Handlers. *Journal of Logical and Algebraic Methods in Programming* 84, 1, 108–123.

[3] Stephen Dolan, KC Sivaramakrishnan, and Anil Madhavapeddy. 2018. Concurrent System Programming with Effect Handlers. In *Trends in Functional Programming*. Springer, 98–117.

[4] Jeff Epstein, Andrew P. Black, and Simon Peyton-Jones. 2011. Towards Haskell in the Cloud. In *Proceedings of the 4th ACM Symposium on Haskell (Haskell '11)*. ACM, 118–129.

[5] Carl Hewitt, Peter Bishop, and Richard Steiger. 1973. A Universal Modular ACTOR Formalism for Artificial Intelligence. *IJCAI* (1973), 235–245.

[6] Daan Leijen. 2014. Koka: Programming with Row Polymorphic Effect Types. In *Proceedings 5th Workshop on Mathematically Structured Functional Programming (MSFP 2014)*. 100–126.

[7] Lightbend Inc. 2023. Akka: Build powerful reactive, concurrent, and distributed applications more easily. https://akka.io.

[8] KC Sivaramakrishnan, Stephen Dolan, Leo White, Tom Kelly, Sadiq Jaffer, and Anil Madhavapeddy. 2021. Retrofitting Effect Handlers onto OCaml. In *Proceedings of the 42nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '21)*. ACM, 206–221.

[9] George Varghese and Tony Lauck. 1987. Hashed and Hierarchical Timing Wheels: Data Structures for the Efficient Implementation of a Timer Facility. *ACM SIGOPS Operating Systems Review* 21, 5 (1987), 25–38.